Architecture — Tourist Safety Monitoring System

This document describes the system as it actually exists in `D:\TouristSafetyMonitoring-src`. Everything here is derived from the source; where a component is *not* verified (e.g. on-device rendering), that is stated explicitly.

---

1. Context

A single Android application (Kotlin + Jetpack Compose) that:

1. subscribes to the platform GPS (Fused Location Provider),
2. evaluates the fix against user-drawn **irregular polygons** using ray casting,
3. records readings and crossings locally (Room/SQLite),
4. raises local notifications, and
5. optionally keeps GPS alive in a **foreground service** with the screen off.

There **is** a backend server and a web dashboard in this repository: `backend/` is a self-contained Flask app over one SQLite file that serves both the phone's API and a static operator dashboard at `/` (see §9). The app talks to it through a real HTTP transport; when no address is configured it falls back to a durable **offline outbox** with an honest stub transport, so nothing is ever falsely reported as delivered (see §6).

---

2. Layer map

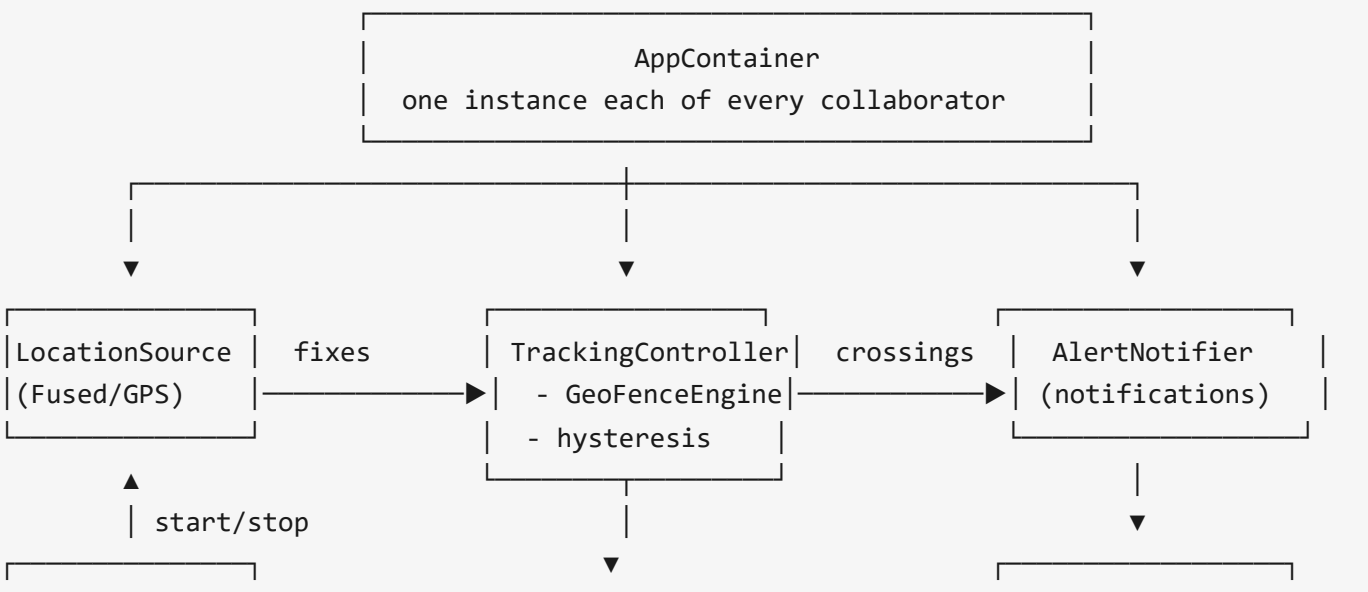
```
app/src/main/java/com/touristsafety/app/
├─ domain/model/         Pure Kotlin. GeoPoint, GeoFenceZone, GeoReading, GeoFenceAlert,
│                        ZoneType, SafetyStatus, LocationSource, OutboxEvent.  (no Android
├─ geofencing/           Pure Kotlin. PolygonGeometry (ray casting), PolygonValidator,
│                        RayCastingContainmentService, GeoFenceEngine (hysteresis state ma
│                        GeoFencePolicy, ZoneStatus, PolygonCodec.            (no Android)
├─ data/
│  ├─ database/          Room: AppDatabase (v3), entities, daos, DatabaseMigrations, Polyg
│  ├─ mapper/            Entity <-> domain mappers (enums stored by stable string keys)
│  └─ repository/        GeoFenceRepository, LocationRepository, AlertRepository, OutboxRe
│                        EmergencyContactRepository
├─ location/             FusedLocationSource (LocationSource iface), LocationRequestSpec,
│                        LocationPermissions, LocationTrackingService (foreground service)
├─ tracking/             TrackingController (real-GPS pipeline), TrackingSnapshot,
```

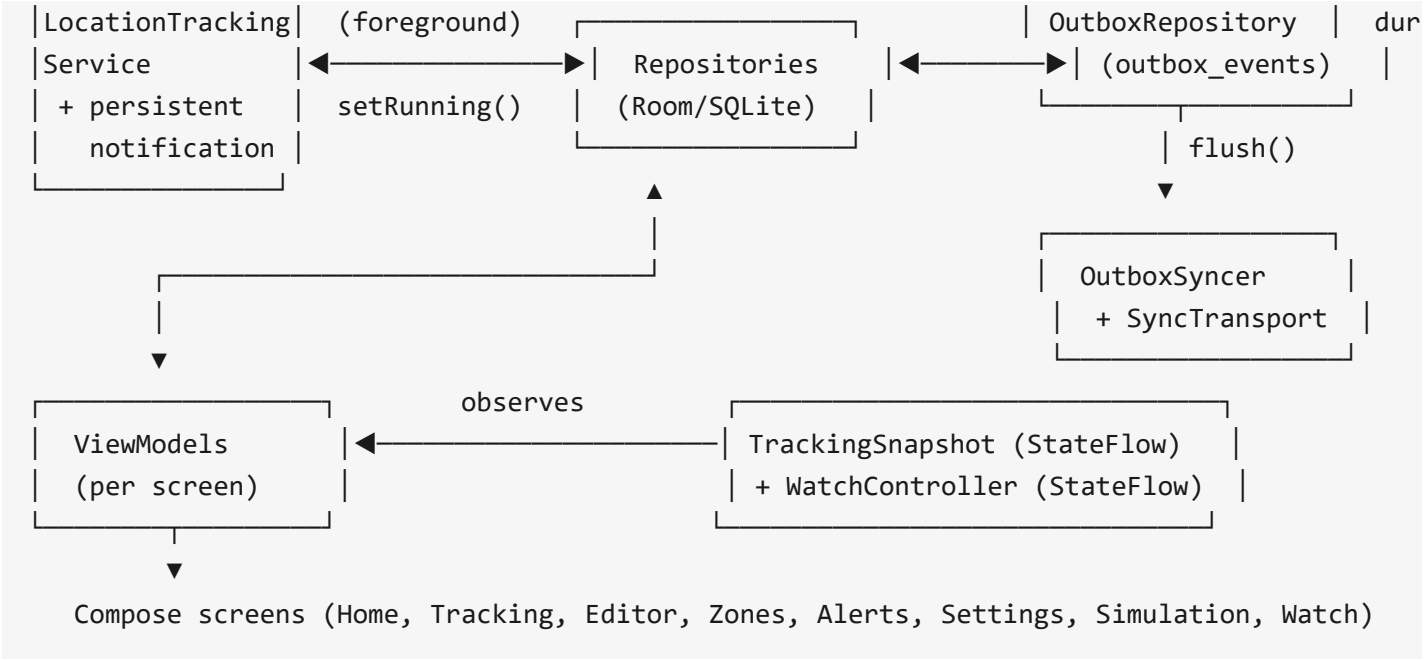
```
│                       BackgroundTrackingControl (start/stop the service)
├─ simulation/          SimulationController, SimulationPathBuilder — quarantined demo da
├─ outbox/              SyncTransport, NoBackendTransport, OutboxSyncer, OutboxPayloads
├─ authority/           AuthorityClient, AuthorityApi (wire codec), AuthorityHttp, Author
│                       (tourist **and** officer sessions, separate keys), AuthoritySyncT
│                       + SelectingSyncTransport, hand-rolled Json
├─ watch/               WatchController, WatchUiState, WatchHaptics, AndroidWatchHaptics
├─ notifications/       NotificationHelper, AlertNotifier, AndroidAlertNotifier
├─ settings/            AppSettings, SettingsRepository (DataStore)
├─ ui/
│  ├─ map/              MercatorProjection (pure, key-less OSM fallback)
│  ├─ components/       SafetyMap (Google Maps path), OfflineOsmMap (key-less path), visu
│  ├─ screens/          Welcome (role chooser), TouristAuth (sign in / create account),
│  │                    Officer (sign-in + register console), Home, Tracking, GeoFenceEdi
│  │                    ManageZones, AlertHistory, Settings, Simulation, Watch,
│  │                    AuthorityConnection (advanced), EmergencyContacts
│  ├─ viewmodel/        One ViewModel per screen + AppViewModelFactory
│  └─ theme/            Fixed green/amber/red safety palette
├─ navigation/          AppNavigation (routes, session-aware start gate, bottom bar, top-
├─ AppContainer.kt      Hand-rolled dependency container (single source of truth)
├─ TouristSafetyApp.kt  Application; owns the container
└─ MainActivity.kt      The only Activity
```

The dependency rule that keeps the core testable: `domain` **and** `geofencing` **depend on nothing Android**. That is why the entire containment/state-machine core runs in plain JVM unit tests.

---

3. Component / data-flow diagram





---

4. The GPS pipeline (real tracking)

1. `TrackingViewModel.startTracking()` checks runtime permission, then asks the Fused provider for updates at the configured interval/accuracy (`AppSettings.requestSpec`).
2. Each fix becomes a `GeoReading(point, accuracyMeters, timestampMillis, source = GPS)`.
3. `TrackingController` evaluates it against every **enabled** zone via `GeoFenceEngine`, which wraps `RayCastingContainmentService`: * accuracy worse than the ceiling → `UNCERTAIN` (decide nothing), * within tolerance of an edge → `BOUNDARY`, * accuracy circle straddles an edge → `UNCERTAIN`, * otherwise ray casting → `INSIDE` / `OUTSIDE`.
4. The engine emits a crossing **only when membership changes**, with hysteresis (exit margin wider than entry margin) so jitter near an edge cannot produce an ENTRY/EXIT ping-pong. `BOUNDARY` and `UNCERTAIN` never flip membership.
5. The controller persists the reading (`LocationRepository`), persists any crossing (`AlertRepository`), raises a notification (`AlertNotifier`) for restricted/dangerous entries, and emits the crossing on `instantAlerts` for the outbox and the watch.
6. A `TrackingSnapshot` `StateFlow` is the single UI-facing state: `isTracking`, last reading, per-zone statuses, overall status, problem flags, and the **true** `foregroundServiceRunning` flag.

Simulation uses a completely separate `SimulationController` that owns its own engine, stamps every row `SIMULATION`, and never touches the Fused provider. The two pipelines never share membership state.

---

5. Map: Google path vs key-less fallback

`SafetyMap` chooses its implementation at runtime:

- **Key configured** (`BuildConfig.MAPS_API_KEY` non-blank) → Google Maps via `maps-compose`, with polygon/polyline overlays (saved zones, the drawn draft, the simulated route, the position fix and its accuracy circle).
- **No key** → `OfflineOsmMap`, a self-contained Compose `Canvas` map that renders **OpenStreetMap raster tiles** through the pure `MercatorProjection`. Tapping still adds vertices, so the Create Geo-Fence screen is fully usable with no key. If tiles cannot be fetched, it degrades to a graticule grid so the screen is never blank.

Containment maths never touches either map implementation.

---

6. Offline outbox and delivery

- Every crossing and every SOS is written to `outbox_events` first (unique `dedupe_key` ⇒ idempotent enqueue ⇒ no duplicates on retry).
- `OutboxSyncer.flush()` reads `PENDING` rows and offers each to a `SyncTransport`. The shipped `SelectingSyncTransport` uses the real `AuthoritySyncTransport` when a base URL is configured and the honest `NoBackendTransport` (transient failure, events stay queued) otherwise. The address is read at flush time, so configuring one takes effect without a restart.
- A background retry loop re-attempts every 30 s. Retries and rejections follow the rules in §9.2: only a permanent rejection discards an event.
- Emergency contacts ride the same outbox (`OutboxKind.CONTACTS`), so an offline edit still syncs.
- Routine **position reports** ride it too (`OutboxKind.POSITION`). `PositionReporter` watches the live snapshot and, when an address is configured, enqueues a throttled real-GPS fix (default 60 s / 100 m; simulated fixes and out-of-range coordinates are refused). The dashboard's "latest position" therefore tracks the tourist between alerts, not only at a crossing.

---

7. Concurrency model

- `AppContainer.appScope` — application-lifetime `SupervisorJob` on `Dispatchers.Default`. Owns the settings observer, the watch driver, the alert→outbox recorder, and the outbox retry loop.
- The `LocationTrackingService` runs on `Dispatchers.Default` and shares the **same** `TrackingController` as the UI (one container), so there is never a second GPS subscription.
- State is exposed as `StateFlow`; screens collect with `collectAsStateWithLifecycle`.

- Room flows are collected on the app scope; the pipeline itself is a suspend function network driven by the location callback.

---

8. Invariants the design protects

1. **A simulated coordinate can never be shown or stored as a real one** — every location and alert row (and every outbox event) carries a `source` / `origin`.
2. **No circular-radius substitute** for containment — ray casting on the real polygon, always.
3. **No duplicate alerts** — membership transitions only, plus hysteresis, plus an idempotent outbox.
4. **No destructive migrations** — additive migrations only; missing migration fails loudly.
5. **The UI never lies about the background service** — status is derived from the service lifecycle.
6. **No fabricated delivery** — with no server configured the watch/SOS status reads "queued", never "delivered"; with a server it echoes the server's actual verdict, and only a permanent rejection discards an event.
7. **A simulated event is never presented as real** — the `simulated` flag travels from the payload into the backend's `events`, `sos_events` and `latest_positions`, and the dashboard badges it.
8. **No control is ever silently dead** — a submit button is gated on *form validity*, never on an unrelated field, and an invalid form names the field it wants. (The bug this replaced: the tourist's sign-in and registration buttons were gated on a backend address the tourist never had to enter.)
9. **The two roles are two sessions, and the boundary is the server's** — the tourist and officer sessions live under separate keys in the same DataStore, so an officer sign-in cannot make the phone behave like a tracked tourist and either sign-out leaves the other alone. The app hides nothing that matters: the officer routes are `require_auth("authority")` on the server, and a tourist token gets **403**.
10. **The app can actually reach its own authority** — the manifest declares a network-security profile that permits **cleartext** HTTP, because the authority is a plain-HTTP server on the LAN and Android 9 (API 28) and later block cleartext by default. Without it, every call dies with `Cleartext HTTP traffic to <host> not permitted` before leaving the phone — looking exactly like a downed server. `NetworkSecurityConfigTest` fails the build if the manifest and the profile drift apart.

8a. The two roles, and where the address lives



```
              │                                  └──── Sign out (clears the tourist only) ──┘
              └──"I'm an authority officer"─▶  Officer  ──▶  register console (GET /author
                                                  │
                                                  └──── Officer sign out (clears the office

          Settings ──▶ Backend connection (advanced)   ← the address, the queue, Test conn
```

`AuthoritySessionStore` holds **two** subjects (tourist and officer) plus the shared base URL, in the `authority_session` DataStore — deliberately *not* `AppSettings`, so signing out can never disturb the tracking tuning. `seedBaseUrlIfUnset(BuildConfig.DEFAULT_AUTHORITY_URL)` runs once in `AppContainer.start()` and is a no-op once an address exists, so a debug build can ship a working default without ever overriding an operator's or tourist's choice. `AuthorityConnectionState` is derived from a real `GET /api/v1/health`, so the screen can distinguish *Connected* from *Offline* instead of reporting success because a string was saved.

8b. Talking to the authority over the LAN (and Android's cleartext policy)

The authority is a **plain-HTTP** server (`http://<lan-host>:8080`); there is no TLS, because it is a single-purpose service on the same private network as the phone. One class sends everything: `HttpUrlConnectionAuthorityHttp`, using the JDK's `HttpURLConnection` on `Dispatchers.IO`. `AuthorityClient` maps any `IOException` (refused, timeout, DNS, TLS) to `AuthorityCall.Unreachable`, which the outbox treats as **transient** — it keeps the event queued rather than discarding it.

Android 9 (API 28) and later **refuse cleartext HTTP by default** for any app whose `targetSdk` is 28+; this app targets API 35. The platform therefore blocks a plain-HTTP request *before it reaches the network*, with the message `Cleartext HTTP traffic to <host> not permitted` — which reads like a dead server but is the phone refusing to send. `app/src/main/res/xml/network_security_config.xml` sets `cleartextTrafficPermitted="true"`, and `AndroidManifest.xml` names it via `android:networkSecurityConfig`. If the authority ever gains TLS, the flag goes back to `false` and the app is pointed at an `https://` address; `NetworkSecurityConfigTest` guards the pairing meanwhile.

---

9. Authority backend (`backend/`)

9.1 Shape

One Flask app (`tsm/app.py`) over one SQLite file (`tsm/db.py`), with no container, no external service and no internet connection required. It serves two audiences: the phone (self-registration, login, `/sync/events` , `/me/contacts`) and the duty officer (a static dashboard at `/` plus the `/authority/*` routes). Accounts are seeded on first start; if no password is supplied through the environment, a random one is generated, printed **once**, and written to `backend/var/DEV_CREDENTIALS.txt` . No password is stored in the source.

A tourist can also **create their own account**: `POST /api/v1/auth/register` (the only public write) mints the Tourist ID from a database sequence (`tourist_id_sequence` , with `tourists.id` as the primary key), hashes the PIN the tourist chose, returns **no secret** and issues **no session**, and reads no privilege field at all — so it cannot mint an operator. The new account appears in the officer's `GET /authority/tourists` register, where **registration**, **tracking** and the safety signals are separate fields and a tourist who has not reported yet shows no position rather than a fabricated one.

9.2 The sync contract

`POST /api/v1/sync/events` takes one entry per queued event and returns one verdict per entry, in order. `DELIVERED` and `DUPLICATE` both mean the authority holds the event (the app treats a duplicate as success); `REJECTED` carries a reason and is permanent. `events` is append-only with a `UNIQUE (tourist_id, dedupe_key)` index — that index makes re-delivery harmless — and `sos_events` is a separate table because an SOS has a workflow (OPEN → ACKNOWLEDGED) that a routine crossing does not.

9.3 Data-flow additions

```
app (AuthorityClient) ──HTTP──▶ /api/v1/auth/register ──▶ tourists (+ tourist_id_seque
                                /api/v1/auth/login ──▶ sessions (expiry, device label
                                /api/v1/sync/events ──▶ events (append-only) ──▶ sos
                                /api/v1/me/contacts  ──▶ emergency_contacts
browser (dashboard)   ──HTTP──▶ /api/v1/authority/*    ──▶ overview · SOS queue · regi
```

Full detail — routes, payload rules, configuration, and how to run it — is in `backend/README.md` .